Certified Kubernetes Administrator (CKA) Exam

Intensive Practice Questions - 150+ Real Exam Simulated Questions

Exam Version: Based on Kubernetes v1.28+

Total Questions: 150+

Time Allocation: 2 hours for 15-20 performance-based tasks

Passing Score: 66%

Format: Performance-based, hands-on tasks in a live Kubernetes environment

Exam Tips & Strategy

- **Time Management:** Allocate ~6-8 minutes per question
- **Use Imperative Commands:** Faster than writing YAML from scratch
- **Kubectl Aliases:** Set up `alias k=kubectl` to save time
- **Documentation Access:** kubernetes.io is available during exam
- **Verify Your Work:** Always check your solution before moving on
- **Skip & Return:** If stuck, flag and move to next question
- **Partial Credit:** Incomplete solutions may earn partial points

Table of Contents

1. [Cluster Architecture, Installation & Configuration (25%)](#section-1)
2. [Workloads & Scheduling (15%)](#section-2)
3. [Services & Networking (20%)](#section-3)
4. [Storage (10%)](#section-4)
5. [Troubleshooting (30%)](#section-5)

Section 1: Cluster Architecture, Installation & Configuration (25%)

RBAC (Role-Based Access Control)

Q1. Create a ServiceAccount named `app-sa` in the `production` namespace. Create a Role named `pod-reader` that allows getting, watching, and listing pods. Bind this role to the ServiceAccount.

Q2. In namespace `dev`, create a ServiceAccount `dev-user`, a Role `pod-manager` with permissions to get, list, watch pods and access pods/log and pods/exec subresources. Create a RoleBinding named `dev-user-binding`.

Q3. Create a ClusterRole named `node-reader` that can get, list, and watch nodes cluster-wide. Create a ClusterRoleBinding named `node-reader-binding` for user `john@example.com`.

Q4. Grant ServiceAccount `monitoring-sa` in namespace `monitoring` cluster-wide permissions to list and get pods in ALL namespaces using appropriate ClusterRole and ClusterRoleBinding.

Q5. In namespace `finance`, create a Role `secret-manager` allowing create, update, delete on secrets. Bind it to group `finance-admins`.

Q6. Check if ServiceAccount `app-sa` in namespace `production` can create deployments. If not, grant the necessary permissions.

Q7. Create a ClusterRole `deployment-manager` with full permissions on deployments and replicasets. Bind to user `admin@example.com`.

Q8. ServiceAccount `logger-sa` in namespace `logging` needs read-only access to ConfigMaps and Secrets across all namespaces. Create appropriate RBAC resources.

Q9. Create a Role in namespace `staging` allowing get, list, watch on pods, services, and endpoints. Bind to ServiceAccount `app-viewer`.

Q10. Verify user `developer@example.com` can create but not delete pods in namespace `dev`. Create necessary RBAC to enforce this.

Cluster Installation with Kubeadm

Q11. Initialize a Kubernetes cluster using kubeadm with pod network CIDR `10.244.0.0/16`, Kubernetes version v1.28.0, and control plane endpoint `k8s-master.local`. Configure kubectl for admin user.

Q12. Generate a kubeadm join command for a worker node. Control plane is at `192.168.1.100:6443`.

Q13. Install Calico CNI plugin and verify all Calico pods are running, nodes show Ready status, and pod-to-pod communication works across nodes.

Q14. Generate a new bootstrap token valid for 24 hours with description "Worker node join token".

Q15. List all kubeadm tokens with expiration times. Delete any expired tokens.

Q16. Create a kubeadm configuration file for initializing a cluster with custom settings: API server advertise address `192.168.1.100`, pod subnet `10.244.0.0/16`, service subnet `10.96.0.0/12`.

High Availability Clusters

Q17. Configure an HA cluster with 3 control plane nodes (192.168.1.101-103), load balancer at 192.168.1.50:6443, pod network CIDR 10.244.0.0/16. Provide initialization steps for all control plane nodes.

Q18. Backup etcd cluster to `/opt/etcd-backup-$(date +%Y%m%d).db`. Etcd runs as static pod with certificates in `/etc/kubernetes/pki/etcd/`. Verify backup integrity.

Q19. Restore etcd from backup `/opt/etcd-backup-20240115.db` to data directory `/var/lib/etcd-restored`. Update etcd manifest and verify cluster functionality.

Q20. Check health of all etcd members: endpoint health, member list with IDs, endpoint status with leader info.

Q21. Remove a failed etcd member from a 3-node cluster. Verify remaining cluster achieves quorum.

Q22. Add a new etcd member to an existing 3-node etcd cluster. Verify the new member joins successfully.

Infrastructure & Prerequisites

Q23. Document all required firewall ports for control plane nodes and worker nodes (incoming and bidirectional).

Q24. Verify containerd is properly installed: check service status, version, test image pull with crictl, list running containers.

Q25. Configure kubelet on worker node with: cluster DNS 10.96.0.10, cluster domain cluster.local, max pods 150.

Q26. Verify system prerequisites on a node: swap disabled, kernel modules loaded (br_netfilter, overlay), sysctl parameters correct, container runtime installed.

Q27. Configure kubelet to use systemd as cgroup driver. Restart kubelet and verify configuration.

Cluster Upgrades

Q28. Upgrade cluster from v1.27.2 to v1.28.0: check upgrade plan, upgrade control plane (kubeadm, kubelet, kubectl), upgrade worker nodes. Provide all commands.

Q29. Upgrade kubelet and kubectl on worker node `worker-1` from v1.27.0 to v1.28.0: drain node, upgrade packages, restart services, uncordon node.

Q30. Before cluster upgrade, verify: current version, available upgrades, component health, etcd backup status.

Q31. Rollback a failed worker node upgrade to previous version. Investigate failure cause.

Q32. Upgrade only the control plane components without upgrading worker nodes. Verify mixed-version cluster operation.

etcd Backup & Restore

Q33. Create automated etcd backup script: snapshot every 6 hours, save to `/backup/etcd/` with timestamp, retain 7 days, log to `/var/log/etcd-backup.log`, alert on failure.

Q34. Disaster recovery test: take snapshot, create test deployment, restore from snapshot, verify test deployment is gone, document recovery time.

Q35. Verify etcd backup at `/backup/etcd-snapshot.db` without restoring: check status, display metadata, list key count.

Q36. Configure etcd to use data directory `/data/etcd` instead of default. Update static pod manifest and verify.

Q37. Take an etcd snapshot and copy it to a remote backup server using scp. Automate this process.

Section 2: Workloads & Scheduling (15%)

Deployments & Updates

Q38. Create Deployment `web-app` in namespace `production`: image nginx:1.21, 3 replicas, labels app=web/tier=frontend/env=production, requests CPU 100m/Memory 128Mi, limits CPU 200m/Memory 256Mi, rolling update maxUnavailable=1/maxSurge=1.

Q39. Update `web-app` deployment to nginx:1.22, record change, monitor rollout, verify all pods updated. If issues occur, rollback.

Q40. Deployment `api-server` has failed rollout. Check status/history, identify problematic revision, rollback to stable version.

Q41. Implement canary deployment: initial deployment 3 replicas v1, canary 1 replica v2, same labels, gradually increase canary if stable.

Q42. Configure Deployment with: minimum ready seconds 30, progress deadline 600s, max unavailable 25%, max surge 25%.

Q43. Pause deployment rollout, make multiple changes (image, env vars, resources), resume rollout. Verify single rollout for all changes.

Q44. Scale `web-app` to 5 replicas using kubectl scale. Then edit deployment to scale to 8. Verify both methods.

Q45. Create a Deployment with revision history limit of 5. Perform 10 updates and verify only 5 revisions are kept.

ConfigMaps & Secrets

Q46. Create ConfigMap `app-config` in namespace `production`: literal values database_url/log_level, from file /opt/config/app.properties, from env file /opt/config/app.env. Use in pod as environment variables.

Q47. Create Secret `db-credentials` with username/password (base64 encoded). Mount as volume at `/etc/db-credentials/` with read-only permissions.

Q48. Create pod using: ConfigMap `app-config` key `database_url` as env `DB_URL`, Secret `db-credentials` keys as env vars, ConfigMap mounted at `/config`.

Q49. Update ConfigMap used by running deployment. Force pods to reload new configuration without manual deletion.

Q50. Create TLS secret from files: tls.crt from /opt/certs/server.crt, tls.key from /opt/certs/server.key, type kubernetes.io/tls.

Q51. Pod cannot start due to missing ConfigMap. Create missing ConfigMap with appropriate data, verify pod starts.

Q52. Create an immutable ConfigMap and Secret. Attempt to modify them and observe the behavior.

Scaling Applications

Q53. Scale deployment `backend-api` from 3 to 8 replicas. Verify all pods running, distributed across nodes, service endpoints updated.

Q54. Configure HorizontalPodAutoscaler for `web-frontend`: min 2, max 10 replicas, target CPU 60%, memory 80%. Generate load and verify autoscaling.

Q55. Scale StatefulSet `database` from 3 to 5 replicas. Verify each pod gets unique PVC, naming follows pattern, scaling is sequential.

Q56. Configure autoscaling based on custom metrics (requests per second). Assume metrics-server and custom metrics API available.

Q57. HPA not scaling deployment. Debug: check metrics server, resource requests, HPA status/conditions, current metrics.

Q58. Create a VerticalPodAutoscaler for a deployment. Configure update mode and resource policies.

Self-Healing Deployments

Q59. Create Deployment with health checks: liveness HTTP GET /health:8080 initial 15s period 10s, readiness HTTP GET /ready:8080 initial 5s period 5s, startup HTTP GET /startup:8080 failure 30 period 10s.

Q60. Create pod with init container: waits for service `database`, checks port 5432 accessible, timeout 60s, then starts main container.

Q61. Create DaemonSet `node-monitor`: runs on all nodes including control plane, image prometheus/node-exporter, mounts host /proc /sys /root, uses hostNetwork and hostPID.

Q62. Create Job: 10 completions, 3 parallel, backoff limit 4, active deadline 600s, command `echo "Processing $HOSTNAME" && sleep 10`.

Q63. Create CronJob: runs daily 2 AM, backs up PVC data, keeps 3 successful/1 failed jobs, deadline 3600s, concurrency policy Forbid.

Q64. Pod keeps restarting due to failed liveness probes. Adjust: initial delay, timeout, failure threshold, period.

Q65. Create StatefulSet for database: 3 replicas, headless service, VolumeClaimTemplate 10Gi, pod management Parallel, update strategy RollingUpdate partition=1.

Q66. Configure a pod with multiple init containers that run sequentially to set up the environment.

Resource Management

Q67. Create namespace `limited` with ResourceQuota: CPU requests 4 cores, memory requests 8Gi, CPU limits 8 cores, memory limits 16Gi, max pods 20, services 10, PVCs 15.

Q68. Create LimitRange in namespace `dev`: default CPU request 100m/limit 500m, memory request 128Mi/limit 512Mi, min CPU 50m/memory 64Mi, max CPU 2/memory 2Gi.

Q69. Pod stuck Pending with "Insufficient cpu". Investigate: node resources, pod requests, ResourceQuota usage, LimitRange constraints. Resolve.

Q70. Calculate if pod (500m CPU, 1Gi memory) can schedule on node: total 2 CPU/4Gi memory, existing pods 1.2 CPU/2.5Gi memory, system reserved 100m CPU/500Mi memory.

Q71. Create PriorityClass `high-priority` value 1000, globalDefault false. Create pod using this priority class.

Q72. Implement pod affinity: prefer nodes with label disktype=ssd, require same zone as backend pods, avoid multiple frontend pods on same node.

Q73. Configure pod with tolerations: tolerate taint special=true:NoSchedule, tolerate node.kubernetes.io/not-ready for 300s.

Q74. Create a pod with resource requests and limits for CPU, memory, and ephemeral storage.

Manifest Management

Q75. Convert imperative command to YAML: `kubectl run nginx --image=nginx:1.21 --port=80 --env="ENV=production" --labels="app=web,tier=frontend" --requests="cpu=100m,memory=128Mi" --limits="cpu=200m,memory=256Mi"`.

Q76. Generate YAML (don't create) for: deployment `api` 3 replicas image api:v1, service exposing port 8080, ConfigMap from directory /opt/config.

Q77. Use `kubectl apply --dry-run=client` and `--dry-run=server` to validate manifests. Explain difference.

Q78. Use `kubectl diff` to preview changes before applying modified manifest. Demonstrate with deployment update.

Q79. Use Kustomize to manage multiple environment configurations (dev, staging, prod) for the same application.

Section 3: Services & Networking (20%)

Network Configuration

Q80. Document cluster network configuration: CNI plugin, pod CIDR, service CIDR, cluster DNS IP, CNI config files location.

Q81. Pod cannot reach other pods. Debug CNI: check plugin pods status, verify configuration, test connectivity, check node routes.

Q82. Create pod using host network: set hostNetwork true, verify pod uses node IP, explain security implications, list use cases.

Q83. Verify IP forwarding enabled on nodes and bridge netfilter module loaded. Fix issues.

Q84. Configure a pod to use a specific network interface or IP address from a multi-homed node.

Pod Connectivity

Q85. Create two pods in namespaces `frontend` and `backend`. Verify communication using: pod IPs, DNS names (pod-ip.namespace.pod.cluster.local), service DNS.

Q86. Debug: Pod A in `frontend` cannot reach Pod B in `backend`. Check NetworkPolicies, DNS resolution, test with curl, check endpoints.

Q87. Demonstrate DNS resolution: service same namespace, service different namespace, service FQDN, headless service pod.

Q88. Pod cannot resolve external DNS (google.com) but resolves internal services. Debug and fix DNS configuration.

Q89. Test and verify pod-to-pod communication across different nodes using ping and curl.

Services & Endpoints

Q90. Create ClusterIP service `backend-service`: selector app=backend/tier=api, port 8080, targetPort 8080, session affinity ClientIP, timeout 3600s.

Q91. Create NodePort service `web-service`: exposes deployment `web-app`, service port 80, target port 8080, NodePort 30080. Verify external access.

Q92. Create LoadBalancer service for `public-api`: port 443, target port 8443, protocol TCP. Explain vs NodePort, verify external IP (cloud).

Q93. Create headless service (ClusterIP None) for StatefulSet `database`: service name database-headless, selector app=database, port 5432. Verify DNS records per pod.

Q94. Manually create Service and Endpoints for external database: service name external-db, external IP 192.168.1.100, port 5432, no selector.

Q95. Service not routing traffic. Debug: check selector matches labels, verify endpoints populated, check pod readiness, test connectivity.

Q96. Create multi-port service: port 80→8080 (HTTP), 443→8443 (HTTPS), 9090→9090 (metrics).

Q97. Configure service with externalTrafficPolicy: Local. Explain the difference from Cluster policy.

Ingress

Q98. Install Nginx Ingress Controller: deploy using official manifests, verify pods running, check ingress class created, test with sample ingress.

Q99. Create Ingress with path-based routing: host app.example.com, path /api→api-service:8080, /web→web-service:80, /admin→admin-service:8080, use TLS secret tls-secret.

Q100. Create Ingress with host-based routing: api.example.com→api-service:8080, web.example.com→web-service:80, admin.example.com→admin-service:8080, each with own TLS.

Q101. Create TLS secret from files: certificate /opt/certs/tls.crt, key /opt/certs/tls.key, secret name app-tls. Configure Ingress for HTTPS.

Q102. Ingress returning 404 errors. Debug: check controller logs, verify ingress config, check backend service/pods, test DNS, verify ingress class.

Q103. Configure Ingress annotations: SSL redirect, connection timeout 60s, proxy body size 10m, rate limiting 10 req/s.

Q104. Create Ingress with default backend serving custom 404 page when no routes match.

Q105. Configure Ingress with URL rewriting and redirect rules.

CoreDNS

Q106. Modify CoreDNS ConfigMap to add custom DNS: database.local→192.168.1.100, cache.local→192.168.1.101. Reload CoreDNS and verify.

Q107. Pod cannot resolve service names. Debug CoreDNS: check pods status/logs, verify service exists, test DNS from pod (nslookup/dig), check /etc/resolv.conf, verify ConfigMap.

Q108. Configure CoreDNS to forward queries for example.com to external DNS 8.8.8.8. Edit ConfigMap, add forward plugin, reload, test.

Q109. Scale CoreDNS to 3 replicas for HA. Configure pod anti-affinity ensuring CoreDNS pods run on different nodes.

Q110. Enable CoreDNS query logging for debugging. Configure log plugin in CoreDNS ConfigMap.

Q111. Configure CoreDNS to use a custom upstream DNS server for all external queries.

Network Policies

Q112. Create NetworkPolicy in namespace `production`: deny all ingress by default, allow ingress from pods with label app=frontend on port 8080, allow from namespace monitoring on port 9090.

Q113. Create NetworkPolicy: allow egress to pods in same namespace, allow egress to kube-dns port 53, deny all other egress.

Q114. Implement NetworkPolicy for three-tier app: frontend can access backend, backend can access database, database accepts only from backend, all tiers access DNS.

Q115. Create NetworkPolicy allowing ingress from IP blocks: allow from 10.0.0.0/8, deny from 10.0.5.0/24.

Q116. NetworkPolicy blocking legitimate traffic. Debug policy rules and test connectivity.

Q117. Create default deny-all NetworkPolicy for namespace, then create specific allow policies for required traffic.

Q118. Configure a NetworkPolicy that allows egress to specific external IP addresses and ports.

Section 4: Storage (10%)

Persistent Volumes

Q119. Create PersistentVolume `pv-data`: capacity 5Gi, access mode ReadWriteOnce, host path /mnt/data, storage class manual, reclaim policy Retain.

Q120. Create StorageClass `fast-storage`: provisioner kubernetes.io/aws-ebs, parameters type=gp3/iopsPerGB=50, volume binding mode WaitForFirstConsumer, reclaim policy Delete.

Q121. List all PVs and identify which are: Available, Bound, Released, Failed.

Q122. PV stuck in Released state. Reclaim it and make available for new claims.

Q123. Create PV with access mode ReadWriteMany, volume mode Filesystem, reclaim policy Recycle. Explain when each access mode should be used.

Q124. Create PV with block volume mode. Demonstrate usage in a pod.

Q125. Change a PV's reclaim policy from Delete to Retain. Explain implications.

Persistent Volume Claims

Q126. Create PVC `data-claim`: requests 2Gi storage, access mode ReadWriteOnce, storage class fast-storage, selector matches label type=ssd.

Q127. Create pod using PVC `data-claim` mounted at /data. Verify persistence: write file, delete pod, recreate, verify file exists.

Q128. PVC stuck Pending. Debug: check available PVs, storage class, access modes, capacity.

Q129. Expand PVC from 5Gi to 10Gi. Verify expansion and pod can use additional space.

Q130. Create a PVC that uses a specific PV by name using a selector.

Application Storage Configuration

Q131. Create StatefulSet `database`: 3 replicas, VolumeClaimTemplate requesting 10Gi storage, each pod gets own PVC, mount path /var/lib/mysql.

Q132. Configure deployment with multiple volumes: ConfigMap at /config, Secret at /secrets, PVC at /data, EmptyDir at /tmp.

Q133. Create pod with init container preparing data in shared volume before main container starts.

Q134. Implement backup strategy for PVCs. Create job backing up data from PVC to external location.

Q135. Configure pod using hostPath volume for logging. Explain security implications and alternatives.

Q136. Create a pod that uses a projected volume combining ConfigMap, Secret, and downward API.

Q137. Configure a StatefulSet with volumeClaimTemplates and verify each pod gets a unique PVC.

Section 5: Troubleshooting (30%)

Cluster & Node Logging

Q138. Node reporting NotReady. Debug: check node conditions, kubelet logs, container runtime status, system resources (CPU/memory/disk).

Q139. Access and analyze logs for: specific pod, previous crashed pod instance, all pods with specific label, kubelet on node.

Q140. Configure kubectl to show logs from multiple containers in a pod simultaneously.

Q141. Pod in CrashLoopBackOff. Investigate using logs, describe, events to identify root cause.

Q142. Kubelet not starting on a node. Check systemd logs, kubelet configuration, and certificate issues.

Application Monitoring

Q143. Check resource usage (CPU/memory) for: all nodes, all pods in namespace, specific pod, containers in pod.

Q144. Identify pods consuming most CPU and memory in cluster. Recommend optimizations.

Q145. Configure resource metrics collection. Verify `kubectl top` commands work.

Q146. Pod being OOMKilled. Analyze issue and adjust resource limits appropriately.

Q147. Set up metrics-server in the cluster and verify it's collecting metrics from all nodes.

Container Logs

Q148. Stream logs from pod in real-time and filter for ERROR messages.

Q149. Retrieve logs from multi-container pod, specifically from sidecar container.

Q150. Configure pod to write logs to both stdout and file in persistent volume.

Q151. Pod logs not appearing. Debug: check container status, log driver config, disk space on node, log rotation settings.

Application Failure Troubleshooting

Q152. Application not responding. Systematically debug: pod status/events, container logs, liveness/readiness probes, resource constraints, network connectivity, ConfigMaps/Secrets.

Q153. Deployment rollout stuck. Identify why new pods aren't starting and fix.

Q154. Pods running but application returns 503 errors. Debug service, endpoints, ingress configuration.

Q155. Application works locally but fails in Kubernetes. Debug environment variables, volume mounts, network policies.

Q156. Pod stuck Pending. Identify and resolve scheduling issue (resources, node selectors, taints/tolerations, affinity rules).

Q157. Application cannot connect to database. Check: service DNS resolution, network policies, database pod status, connection credentials from secrets.

Q158. Application experiencing intermittent failures. Debug and identify the root cause.

Cluster Component Failure

Q159. API server not responding. Debug: check API server pod/process status, logs, etcd connectivity, certificate validity, port availability.

Q160. Scheduler not scheduling pods. Investigate: scheduler pod status, logs, API server connectivity, node conditions.

Q161. Controller manager not creating replica pods. Debug controller manager and verify API server communication.

Q162. Kubelet on worker node not registering with cluster. Check: kubelet service status, configuration, certificates, API server connectivity.

Q163. CoreDNS pods not running. Debug and restore DNS functionality.

Q164. kube-proxy not updating iptables rules. Debug service connectivity issues related to kube-proxy.

Q165. etcd cluster unhealthy. Check member status, logs, restore from backup if necessary.

Q166. Control plane components failing after certificate expiration. Renew certificates and restart components.

Network Troubleshooting

Q167. Pods cannot communicate across nodes. Debug: CNI plugin status, network routes, firewall rules, pod CIDR configuration.

Q168. Service not load balancing traffic to pods. Debug: service endpoints, pod labels/selectors, pod readiness, kube-proxy rules.

Q169. Ingress returning 404 errors. Debug: ingress controller logs, ingress resource config, backend service, path routing rules.

Q170. DNS resolution failing for services. Debug: CoreDNS pods, CoreDNS service, pod /etc/resolv.conf, network policies affecting DNS.

Q171. External traffic cannot reach NodePort services. Check: node firewall rules, security groups (cloud), kube-proxy config, service config.

Q172. NetworkPolicy blocking required traffic. Identify blocking policy and create exception.

Q173. Pods can reach services by IP but not by DNS name. Debug DNS configuration and CoreDNS.

Q174. Intermittent network connectivity issues between pods. Debug and identify the root cause.

Advanced Troubleshooting Scenarios

Q175. Complete application stack (frontend/backend/database) not working. Systematically debug: all pod statuses, service connectivity, ingress routing, ConfigMaps/Secrets, network policies, persistent storage.

Q176. After cluster upgrade, several applications stopped working. Identify and fix: API version deprecations, changed default behaviors, broken RBAC rules, CNI plugin compatibility.

Q177. Node experiencing high load. Identify resource-intensive pods and implement: resource limits, pod priority classes, node affinity rules, eviction policies.

Q178. Implement complete monitoring solution: deploy metrics-server, configure resource quotas, set up HPA, create alerts for resource exhaustion.

Q179. Secure namespace by implementing: network policies (default deny), pod security standards, RBAC restrictions, resource quotas.

Q180. Migrate stateful application from one node to another without data loss: cordon source node, drain safely, verify PV attachment, validate application state.

Q181. Debug certificate expiration affecting: API server, kubelet, service accounts, admission webhooks.

Q182. Troubleshoot a service mesh (Istio) integration: install Istio, enable sidecar injection, configure traffic routing, debug mTLS issues.

Q183. Perform disaster recovery drill: simulate control plane failure, restore from etcd backup, verify cluster state, validate application functionality.

Q184. Optimize cluster performance: identify bottlenecks, tune kubelet settings, optimize etcd, configure node resources.

Q185. Debug persistent volume provisioning issue in cloud environment: check storage class, verify cloud credentials, check IAM permissions, validate volume plugin.

Q186. Troubleshoot complex networking with multiple network policies, ingress rules, and service mesh configurations.

Q187. Debug custom scheduler issues: deploy custom scheduler, configure pod to use it, verify scheduling decisions, troubleshoot failures.

Q188. Perform complete cluster health check: verify all components, check resource utilization, validate security posture, optimize performance, document findings.

Q189. Application pods evicted due to node pressure. Investigate eviction reasons and implement preventive measures.

Q190. Debug issues with pod security policies/standards preventing pod creation.

Exam Day Checklist

Before the Exam

• [ ] Verify system requirements (Chrome browser, stable internet)

• [ ] Test webcam and microphone

• [ ] Clear workspace (no papers, phones, extra monitors)

• [ ] Have government-issued ID ready

• [ ] Review kubectl cheat sheet

• [ ] Practice with time constraints

During the Exam

• [ ] Read each question carefully

• [ ] Note the weight of each question

• [ ] Use imperative commands when possible

• [ ] Verify your work before moving on

• [ ] Flag difficult questions and return later

• [ ] Keep track of time

• [ ] Use kubernetes.io documentation effectively

Key kubectl Commands to Remember

```
# Set alias
alias k=kubectl

# Generate YAML
k run nginx --image=nginx --dry-run=client -o yaml
k create deployment nginx --image=nginx --dry-run=client -o yaml
k expose deployment nginx --port=80 --dry-run=client -o yaml

# Quick operations
k get pods -A
k describe pod
k logs -f
k exec -it -- /bin/sh
k delete pod --force --grace-period=0

# Debugging
k get events --sort-by='.lastTimestamp'
k top nodes
k top pods
k auth can-i --as=

# YAML manipulation
k explain pod.spec
k explain deployment.spec.strategy
```

Common Troubleshooting Steps

1. Check pod status: `kubectl get pods`

2. Describe pod: `kubectl describe pod `

3. Check logs: `kubectl logs `

4. Check events: `kubectl get events`

5. Verify service endpoints: `kubectl get endpoints`

6. Test DNS: `kubectl run test --image=busybox -it --rm -- nslookup `
7. Check RBAC: `kubectl auth can-i `

Additional Practice Resources

- **Official Kubernetes Documentation:** https://kubernetes.io/docs/

- **Kubectl Cheat Sheet:** https://kubernetes.io/docs/reference/kubectl/cheatsheet/

- **Practice Environments:**

• Killercoda (free interactive scenarios)

• Killer.sh (CKA simulator - 2 sessions included with exam)

• Minikube/Kind for local practice

Good luck with your CKA exam preparation!

Remember: The CKA exam tests practical skills. Practice in a real Kubernetes environment as much as possible.

Document Version: 1.0

Last Updated: February 2024

Total Questions: 190

Estimated Study Time: 40-60 hours of hands-on practice